Real-time Game Leaderboard — Database Portfolio

MIT 8103 Advanced Database Systems · Taiwo Oyewale · Student ID 301926988

GitHub repository (https://github.com/TaiwoSaidat/Oyewale_Taiwo_301926988) · Final summary (Oyewale_Taiwo_301926988_FinalSummary.md) · Final summary (PDF) (Oyewale_Taiwo_301926988_FinalSummary.pdf)

Purpose and case study

This portfolio designs and evaluates the database layer for a **Real-time Game Leaderboard**. Players register, submit scores for a game and season, and appear in rankings. Viewers read leaderboards, while operators review submissions and corrections.

The relational design stores three entities: `player` (account handle and display name), `game` (title, mode and active status) and `score_entry` (one score submission, with player and game foreign keys, season and an idempotency key). The NoSQL part adds `leaderboard_events`, an append-only event log for score submissions and corrections with embedded player/game snapshots. The portfolio supports these operations:

- register players and games;
- record score submissions without duplicate retries;
- filter and rank scores by game and season;
- record score-submitted and score-corrected events;
- discuss a resilient, distributed deployment for the event log.

All records are fictional and were created for this assessment. Portfolio 5 is a **proposed design/discussion only**; no cloud cluster was deployed or tested.

Portfolio contents

Portfolio	Focus	Main evidence	README
1 — Database design and modelling	Relational schema, keys, constraints, normalisation	Schema, sample data, 9 rejected invalid inserts, ER diagram	Portfolio 1 README (portfolio-1-database-design/README.md)
2 — Query processing and optimisation	Joins, aggregation, execution plans, indexing	Query results, before/after plans, 10,000-row index comparison	Portfolio 2 README (portfolio-2-query-optimisation/README.md)

Portfolio	Focus	Main evidence	README
3 — Transactions and concurrency	COMMIT, ROLLBACK, READ COMMITTED, row locking	Commit/rollback trace, two-session lost-update comparison	Portfolio 3 README (portfolio-3-transactions-concurrency/README.md)
4 — NoSQL and advanced data models	Document modelling, validation, queries	2 event documents, 4 queries, 2 rejected writes	Portfolio 4 README (portfolio-4-nosql/README.md)
5 — Distributed and cloud databases	Replication, consistency, sharding, recovery	Proposed replica-set architecture and failure analysis (not executed)	Portfolio 5 README (portfolio-5-distributed-cloud/README.md)

Assessment coverage

Area	Where to find the evidence
Database design and modelling	Portfolio 1 README (portfolio-1-database-design/README.md) · design (portfolio-1-database-design/Oyewale_Taiwo_301926988_P1_Design.md) · ER diagram (portfolio-1-database-design/Oyewale_Taiwo_301926988_P1_ERDiagram.md) · normalisation (portfolio-1-database-design/Oyewale_Taiwo_301926988_P1_Normalisation.md)
Query processing and optimisation	Portfolio 2 README (portfolio-2-query-optimisation/README.md) · queries (portfolio-2-query-optimisation/Oyewale_Taiwo_301926988_P2_Queries.sql) · index experiment (portfolio-2-query-optimisation/Oyewale_Taiwo_301926988_P2_Optimisation.sql)
Transactions and concurrency	Portfolio 3 README (portfolio-3-transactions-concurrency/README.md) · commit/rollback (portfolio-3-transactions-concurrency/Oyewale_Taiwo_301926988_P3_Transactions.sql) · Session A (portfolio-3-transactions-concurrency/Oyewale_Taiwo_301926988_P3_SessionA.sql) · Session B (portfolio-3-transactions-concurrency/Oyewale_Taiwo_301926988_P3_SessionB.sql)
NoSQL and advanced data models	Portfolio 4 README (portfolio-4-nosql/README.md) · MongoDB script (portfolio-4-nosql/Oyewale_Taiwo_301926988_P4_MongoDB.js)
Distributed and cloud databases	Portfolio 5 README (portfolio-5-distributed-cloud/README.md) · proposed architecture (portfolio-5-distributed-cloud/screenshots/Oyewale_Taiwo_301926988_P5_ProposedDesign.png)
Individual reflection and AI-use declaration	Final summary (Oyewale_Taiwo_301926988_FinalSummary.md)

Tools and versions

Tool	Version used for the evidence
PostgreSQL server and `psql`	PostgreSQL 17 / psql 17

Tool	Version used for the evidence
MongoDB server	MongoDB 7
MongoDB shell	`mongosh` 2.x
Diagram source	Mermaid
Version control and evidence hosting	Git, GitHub

No cloud service, replica set, shard cluster or managed database was provisioned. Portfolio 5 therefore contains reasoning and a proposed diagram, not operational evidence.

How to navigate and reproduce

The portfolios build on each other in this order:

1. **Portfolio 1** creates the `portfolio` schema and loads the small sample dataset. Start here.
2. **Portfolio 2** reuses the Portfolio 1 database and adds a separate `query_lab` schema for the benchmark.
3. **Portfolio 3** reuses the Portfolio 1 database, adds a temporary counter column and needs two `psql` sessions.
4. **Portfolio 4** is independent of PostgreSQL and needs a fresh MongoDB database.
5. **Portfolio 5** has nothing to execute; it is a design discussion.

Each portfolio README contains its own prerequisites, exact run order, expected output and limitations. Use a **fresh database per portfolio group** and follow the commands from inside the relevant portfolio folder. Timings in Portfolio 2 are local illustrative measurements, not production benchmarks, and the Portfolio 5 design was not executed.

Repository layout

```
.
├── README.md                                # this landing page
├── Oyewale_Taiwo_301926988_FinalSummary.md/.pdf
├── images/                                  # ER diagram used by Portfolio 1
├── portfolio-1-database-design/             # schema, sample data, tests, design docs
├── portfolio-2-query-optimisation/          # queries, benchmark script, plans
├── portfolio-3-transactions-concurrency/    # commit/rollback and two-session scripts
├── portfolio-4-nosql/                       # MongoDB model, script and results
└── portfolio-5-distributed-cloud/           # proposed architecture discussion
```

Each portfolio folder has a `README.md`; captured images are in each folder's `screenshots/` directory, except the Portfolio 1 ER image, which lives in `images/`. Figure captions state what each image demonstrates.

AI-use declaration

I used GitHub Copilot and a portfolio generator to help prepare the database model, scripts and explanations. The generator ran the PostgreSQL and MongoDB exercises, checked their outputs and captured the result images. I remain responsible for checking and explaining the submitted work. I re-ran each portfolio's commands in a fresh environment, compared the output with the README tables, corrected the two inconsistencies I found, and checked all links and claims against the scripts The full declaration is in the final summary (Oyewale_Taiwo_301926988_FinalSummary.md#ai-use-declaration).

References

The full reference list is in the final summary (Oyewale_Taiwo_301926988_FinalSummary.md#references). Key sources:

- PostgreSQL 17: constraints (https://www.postgresql.org/docs/17/ddl-constraints.html)
- PostgreSQL 17: EXPLAIN (https://www.postgresql.org/docs/17/using-explain.html)
- PostgreSQL 17: transaction isolation (https://www.postgresql.org/docs/17/transaction-iso.html)
- PostgreSQL 17: explicit locking (https://www.postgresql.org/docs/17/explicit-locking.html)
- MongoDB: data modelling (https://www.mongodb.com/docs/manual/data-modeling/)
- MongoDB: replication (https://www.mongodb.com/docs/manual/replication/)
- MongoDB: sharding (https://www.mongodb.com/docs/manual/sharding/)